

Python/Rによる データ解析ハンズオン

日本行動計量学会 第2回ECR支援セミナー

2026年8月8日（土）－ 9日（日） @ 電気通信大学

神沢雄大 Yuta Kanzawa

おことわり

- 本コースの資料（スライド、演習教材、プログラム）は受講者ご本人に限ってご利用下さい。
 - 営利・非営利を問わず本コースの資料の全部または一部のご利用をご希望の場合はセミナー事務局までお問い合わせ下さい（著作権法第35条に該当する場合は除く）。
- 本コースの資料の全部または一部の無断複製・転載を不可とします（営利・非営利を問わず）。
 - 複製・転載をご希望の場合はセミナー事務局までお問い合わせ下さい（著作権法第32条第1項に該当する場合は除く）。

手を動かして学ぶ!

第2回 ECR支援セミナー

Python / R

による
データ解析ハンズオン

2日間 でデータ解析の基礎を体系的に学ぶ!

```
# Plotlyでデータを可視化
import plotly.express as px
import pandas as pd
df = pd.read_csv("data.csv")
fig = px.scatter(df, x="x", y="y",
                color="group", size="value")
fig.show()
```

環境構築 (PythonとRの導入)

データ加工

データ可視化 (Plotly)

統計解析

機械学習

ベイズ統計

2026年 8月8日(土) 10:30~18:30

8月9日(日) 9:00~12:40頃 午前のみ

電気通信大学 (東京都調布市市田ヶ丘1-5-1) ※建物・教室は後日案内

対面+オンライン・ライブ配信 (オンラインは聴講のみ)

大学院生・若手研究者 (ECR) 対象!

<https://peatix.com/event/5046999>

スケジュール（全9コマ）

8月8日（土）

時間	内容
10:30-10:40	オープニング
10:40-11:40	Ex1 環境構築
11:40-12:40	ランチ
12:40-13:40	Ex2 文法（基礎）
13:50-14:50	Ex3 文法（応用）
14:50-15:10	コーヒースタイル
15:10-16:10	Ex4 データ加工（基礎）
16:20-17:20	Ex5 データ加工（応用）
17:30-18:30	Ex6 データ可視化

8月9日（日）

時間	内容
9:00-9:10	オープニング
9:10-10:10	Ex7 統計解析
10:20-11:20	Ex8 機械学習（回帰・分類）
11:30-12:30	Ex9 ベイズ統計
12:30頃	クロージング

コースの進め方

各コマ: 要点 → 簡易サンプル → python/ で実装
正解は solutions/。R / Stanは対訳

前提・時間・Must / Stretch

- 表形式データの分析を通じてPythonを習得する（Rは対訳）
- 説明 → 演習（Must+Stretch） → Q&A
 - Must: 全員
 - Stretch: 余力がある人（説明はします）

リポジトリとオンライン

- リポジトリ :
https://github.com/ytknzw/bms_ecr_seminar_202608.git
 - 詳細は後ほど説明。
- フォルダー構成
 - exercises/.../python/ ← 問題
 - exercises/.../solutions/ ← 正解
 - exercises/.../r/ ← Rでの実装

はじめに

- 生成AIツールによる**実装コストの低下**
- 科学的な用途のプログラムは**コード自体の正確性や信頼性が重要**
→ 人間が吟味
- **自分で読み書きできることが必要**

Exercise 1

環境構築（Pythonの導入）

学習目標 (Must)

- 公開リポジトリを git clone し、リポジトリを操作できる。
- uvでPythonを実行できる。
- 必須パッケージの確認ができる。

プログラミング言語とは

- コンピューターが実行すべき**処理を記述**する言語（≠自然言語）
 - コンピューターが直接理解し実行できる命令からなる言語：機械語
- 有名なもの（Python以外）
 - Java、C、C++、JavaScript、Ruby、Julia、Go、Rust、R、SQL
- **インタープリター言語とコンパイラ言語**
 - インタープリター言語：ソースコードを**逐次機械語に翻訳**しながら実行。
 - 実行に時間が掛かるが、変更が楽。
 - コンパイラ言語：ソースコードを一括して**機械語に翻訳**（別途実行）。
 - 変更の手間が掛かるが、実行が早い。

Pythonとは



- 「パイソン」
- 1991年に開発された**プログラミング言語**
 - Python = ニシキヘビ
- 最初の開発者：ガイド・ファン・ロッサム
 - コメディアングループ「モンティ・パイソン」から命名
- 最新バージョン：**3.14.7**（2026年8月5日リリース）
- 特徴：処理の対象範囲や順序をインデントで表す。
- 本講座ではPython 3.13を用いる。

ライブラリーとは

- ライブラリー：何らかの機能の集合（概念）
 - パッケージ：その機能を実装したもの（実体）
- プログラミング言語の**標準機能にない新たな機能を提供。**
- **あらかじめインストールしておき、使うときに呼び出す（読み込む）。**
- Python
 - 標準機能*（Pythonそのもの）
 - 標準モジュール*（Pythonとセットで公式に提供されているライブラリー）
 - インストール不要だが呼び出す必要あり。
 - **サードパーティー製パッケージ（ライブラリー）**：PyPI、GitHubで公開。

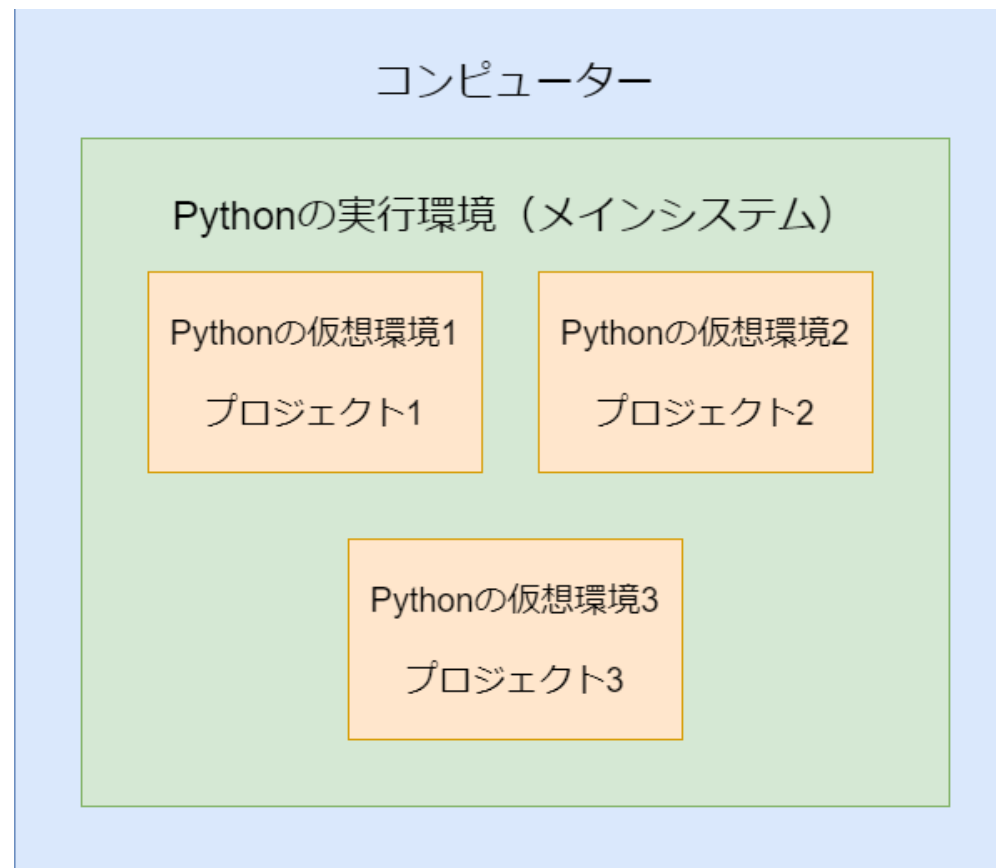
* 標準ライブラリー（標準機能と標準モジュール）の一覧：<https://docs.python.org/ja/3.14/library/>

PythonとRのライブラリー

Python	主な用途	R
pathlib	パス操作（標準・Ex2）	fs / base
pytest	テスト（Ex3）	testthat
pandas	表データの読込・加工・集計（Ex4）	tidyverse（dplyr 等）
numpy	配列・数値計算（Ex4）	(base)
pyarrow	parquet の読み書き（Ex4）	arrow
requests	Web取得（Ex4 Stretch）	httr / httr2
plotly	可視化（Ex6）	ggplot2
statsmodels	統計・回帰（Ex7）	broom / stats
scikit-learn	機械学習（Ex8）	tidymodels
lightgbm	機械学習（Ex8）	lightgbm
pymc	ベイズ推論（Ex9）	cmdstanr / Stan

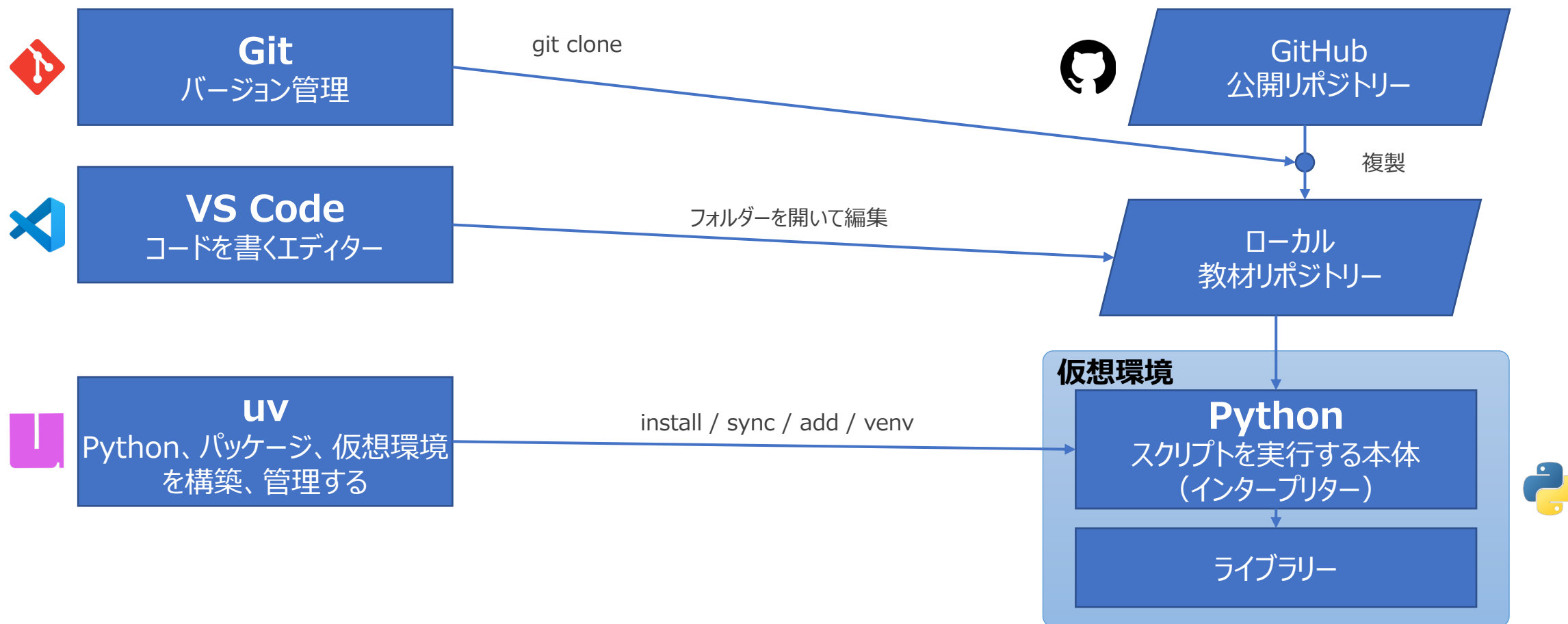
仮想環境

- 用途（プロジェクト）に応じた必要最小限のライブラリーを使う
 - ライブラリー間の依存関係
 - プロジェクトごとに環境を隔離し、依存の衝突を防ぐ
- パッケージ管理ツール
 - uv（他に pip, venv, conda など）



* https://pycamp.pycon.jp/_images/venv-image.png

演習1：環境構築



* □ ソフトウェア □ 読み書きするファイル (リポジトリ)



演習1：環境構築

- Visual Studio Code (VS Code) をインストールします。
 - ダウンロード: <https://code.visualstudio.com/>
 - インストール (macOS) :
 - <https://code.visualstudio.com/docs/setup/mac>
 - インストール (Windows) :
 - <https://code.visualstudio.com/docs/setup/windows>
 - 拡張機能: Python (Microsoft) を入れる
 - <https://marketplace.visualstudio.com/items?itemName=ms-python.python>
 - 拡張機能: Jupyter (Microsoft) を入れる (Ex6の .ipynb 用)
 - <https://marketplace.visualstudio.com/items?itemName=ms-toolsai.jupyter>



演習1：環境構築

- Gitをインストールします。
 - インストール（macOS）：
 - brew install git
 - または xcode-select --install（Command Line Tools）
 - または <https://git-scm.com/download/mac>
 - インストール（Windows）：
 - <https://git-scm.com/download/win> のインストーラー
 - 確認：git --version
- 教材リポジトリをGitHubから取得します（書き込み可能な場所で）。
 - git clone https://github.com/ytknzw/bms_ecr_seminar_202608.git



演習1：環境構築

- uvをインストールします。
 - macOS
 - `curl -Lsf https://astral.sh/uv/install.sh | sh`
 - https://docs.astral.sh/uv/getting-started/installation/#__tabbed_1_1
 - Windows
 - `powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"`
 - https://docs.astral.sh/uv/getting-started/installation/#__tabbed_1_2
- 確認：`uv --version`



演習1：環境構築

- Pythonのインストール：`uv python install 3.13`
 - もしくはPython公式サイトからインストーラーをダウンロード。
- 仮想環境構築（作業フォルダー内にて）
 - `cd <REPO>`
 - 本講座リポジトリでは`uv sync / uv run`でも可（SETUP.md）
 - プロジェクト初期化：`uv init`
 - `$env:Path = "C:\Users\ユーザー名\.local\bin;$env:Path"`
 - 仮想環境作成：`uv venv --python 3.13`
 - 仮想環境有効化：`.venv\Scripts\activate`
 - `Set-ExecutionPolicy Unrestricted -Scope CurrentUser`

Ex1 — 演習（実装）

- SETUP.mdに沿って環境を構築する。
- 動作確認
 - `uv run python env/check_packages.py`

Exercise 2

文法（基礎）

学習目標（Must）

- 変数・型・演算子・import を読み書きできる
- リストと辞書（イテラブル）を扱える
- 条件分岐・繰り返し・小さな関数定義を書ける

Python言語の基礎

Python language 101

コードスタイルガイド

- コードを書く際の様式のルール（名前、空白、改行、行の長さ、など）
 - 書式が整ったコードは読みやすい → デバッグや保守をしやすい
 - 意識しながら書くのは大変 → フォーマッターやリンターの使用
- PEP 8（Pythonの公式スタイル）：
<https://peps.python.org/pep-0008/>
- R: tidyverse style guide（
<https://style.tidyverse.org>） — styler / lintr の土台

* Google Python Style Guide: <https://google.github.io/styleguide/pyguide.html>

フォーマッターやリンターの使用

項目	Python (Ruff)	R (参考)
役割	見た目の整形・静的解析	見た目の整形・静的解析
フォーマッター	ruff format (おすすめ)	styler (tidyverse style 準拠)
リンター	ruff check (おすすめ)	lintr
運用	フォーマットとリンターをruffにまとめることが多い	RStudio アドインでも実行可能
例 (リンター)	未使用変数、曖昧な比較、危険なデフォルト引数	バグにつながりやすい書き方のチェック

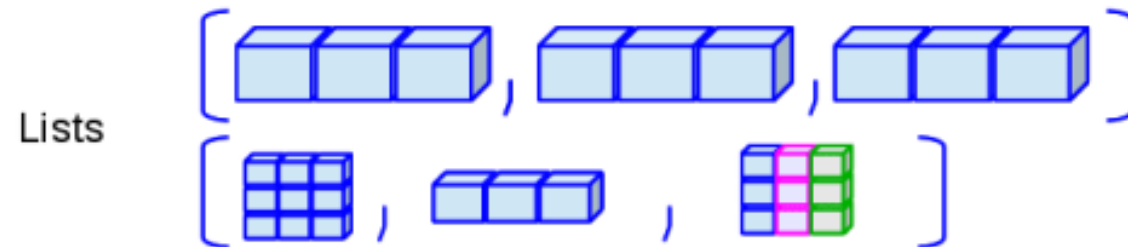
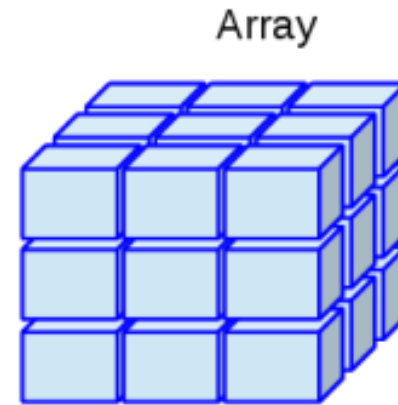
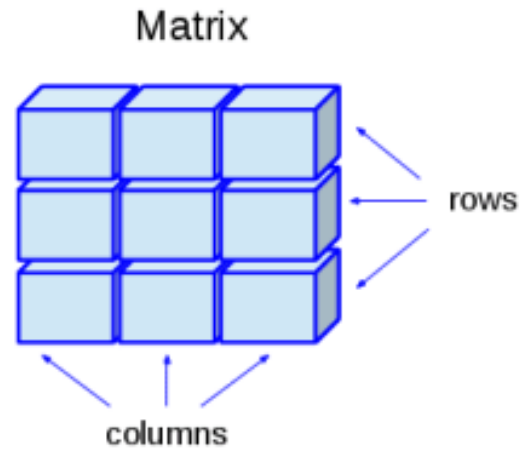
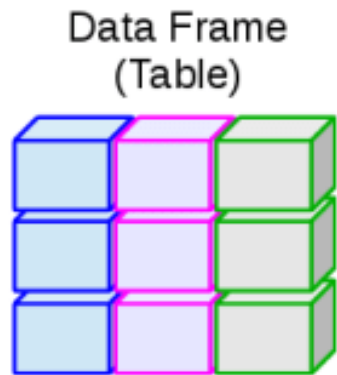
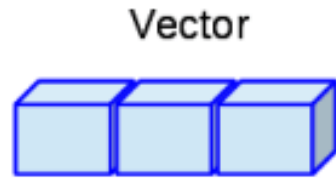
* 参考: Pythonのフォーマッター (black / isort) 、リンター (Flake8) など。R: <https://github.com/r-lib/styler> , <https://github.com/r-lib/lintr>

データ型

データ型	Python	R
文字列	str	character
整数	int	integer
数値（浮動小数点数）	float	numeric
論理値	bool (True, False)	logical (TRUE, FALSE)
日付	datetime.date	Date
日時	datetime.datetime	POSIXct / POSIXlt
カテゴリー	pandas.Categorical	factor
欠測値（欠損値）	None / pd.NA / np.nan	NA

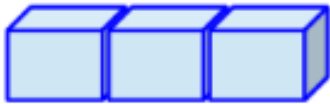
* 型は type(x) や df.dtypes で確認

Rのデータ構造

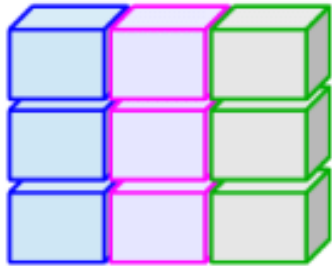


Pythonのデータ構造

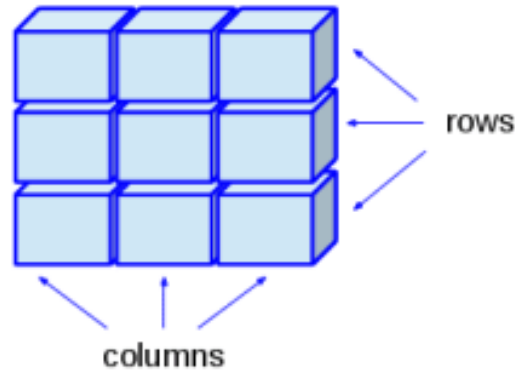
list / pandas.Series



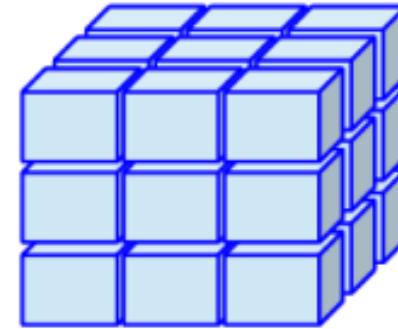
pandas.DataFrame



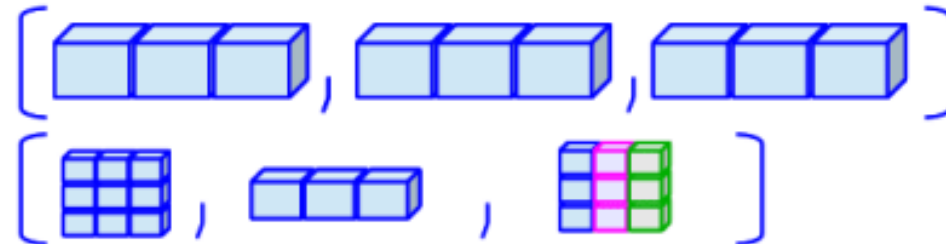
numpy.ndarray (2D)



numpy.ndarray (nD)



list (入れ子可)



その他、dict、tuple、set

データ構造の比較

種類	Python (pandas)	R
1次元の値の並び (順序あり)	list / Series 例: [1, 2, 3]	vector / list
表 (行×列)	DataFrame	data.frame / tibble
キーと値の組の並び (順序あり)	dict (辞書) 例: {"a": 1, "b": 2}	named list
変更不可の並び (順序あり)	tuple (タプル) 例: (1, 2, 3)	(対応なし。list で代用)
重複なしの集合 (順序なし)	set (集合) 例: {1, 2, 3}	(対応なし。unique() 等)

* 集合は積集合や和集合など集合の演算が可能 (割愛)。

演算子

区分	Python	意味	R
算術	+ - * /	加法・減法・乗法・除法	(同じ)
算術	** // %	累乗・商・剰余	^ %/% %%
比較	== !=	等しい・等しくない	(同じ)
比較	> < >= <=	超・未満・以上・以下	(同じ)
比較	in	含まれる	%in%
論理	and or not	真偽値の結合	& ! (要素) / && (スカラー)
ビット	& ~	ビット演算	bitwAnd / bitwOr / bitwNot 等

オブジェクトに値を割り当てる

項目	Python	R
代入	「=」	「<-」 (「=」も可)
例	<code>a = 1 / df = pd.read_csv(...)</code>	<code>a <- 1 / df <- read_csv(...)</code>
複合代入	<code>a += 1</code> (<code>a = a + 1</code> と同じ)	(同じ演算子はなし。 <code>a <- a + 1</code>)
名前に使える文字	英字、数字、「_」 (先頭は数字不可)	Rも同じ
上書き	警告なし	Rも同じ

f-文字列（フォーマット済み文字列リテラル）

- 文字列リテラル（コードの文字列そのもの）の直前にfを付ける。
- 中括弧 {式} に変数や式を書いて文字列に埋め込む。
 - %書式やstr.format()より読みやすい。
 - 例: `f"hello {name}! 2n={n*2}"` → `hello world! 2n=4`
- 式の末尾に「=」を付けると、式のテキスト（オブジェクト名など）と値の両方が展開され便利（特にデバッグ時）。
 - 例: `f"{n*2=}"` → `n*2=4`
 - 「=」がないと「4」と出力される。

* R: glue::glue() / sprintf() に近い

Pythonのイテラブル

- イテラブル: 1つ以上の要素を持ち、それを1つずつ取り出せるもの

種類	記法・例	特徴
リスト (配列)	<code>[]</code> で挟み、コンマで区切る。重複は OK 例: <code>[1, 2, 3, 4, 5]</code> 、 <code>["Alice", "Bob", "Carol"]</code>	値を並べたもの <code>sorted(xs)</code> は新しいリスト、 <code>xs.sort()</code> は破壊的
タプル	<code>()</code> で挟み、コンマで区切る。要素の追加・代入は不可 例: <code>(1, 2, 3)</code> 、 <code>("Alice", "Bob")</code>	順序あり・変更できない並び (イミュータブル)
集合 (set)	<code>{}</code> で挟む (空は <code>set()</code>)。重複は自動除去 例: <code>{1, 2, 3}</code> 、 <code>set([1, 1, 2]) → {1, 2}</code>	重複のない要素の集まり 順序はあてにしない
辞書	<code>{}</code> で挟み、キーと値はコロンで区切る。キーの重複は不可 例: <code>{"ユーザー1": 2, "ユーザー4": 0, "ユーザー2": 3}</code>	キーと値のペアの配列

イテラブルの操作

- lはリスト、tはタプル、sは集合、dは辞書とする

操作	例	説明
参照 (list / tuple)	<code>l[0] / l[-1]</code> <code>l[1:3]</code> # 末尾のインデックスは含まれない。 <code>t[0]</code>	インデックスは0始まり（末尾は-1） タプルも参照・スライスは可
リストの追加・削除	<code>l.append(x)</code> <code>l.pop()</code> / <code>l.remove(x)</code>	<code>append</code> は末尾へ（破壊的） <code>pop</code> は戻り値あり、 <code>remove</code> は値指定
タプル（変更不可）	# <code>t.append(x)</code> 不可 # <code>t[0] = y</code> も不可	追加・削除・要素代入はできない （イミュータブル）
集合 (set)	<code>s.add(x)</code> <code>s.discard(x)</code> / <code>s.remove(x)</code>	インデックス参照は不可 <code>discard</code> は無い要素でもエラーなし
辞書	<code>d[k] / d.get(k)</code> <code>d[k] = v</code> <code>d.keys()</code> / <code>d.items()</code>	<code>[]</code> はキーなしでエラー、 <code>get</code> は安全 代入で追加・更新、 <code>items</code> で走査

繰り返し (for)

項目	Python	R
構文	<pre>for x in <i>iterable</i>: ...</pre>	<pre>for (x in <i>iterable</i>) { ... }</pre>
例	<pre>for n in range(1, 11): print(n ** 2)</pre>	<pre>for (n in 1:10) { print(n ^ 2) }</pre>
インデント	字下げでブロックの範囲を表す 半角スペース4文字がよい (PEP 8)	波括弧 {} でブロックを表す

繰り返し (while)

項目	Python	R
構文	<pre>while <i>condition</i>: ...</pre>	<pre>while (<i>condition</i>) { ... }</pre>
例	<pre>total = 0 i = 1 while i <= 10: total += i i += 1</pre>	<pre>total <- 0 i <- 1 while (i <= 10) { total <- total + i i <- i + 1 }</pre>

条件分岐 (if)

項目	Python	R
構文	<pre>if <i>condition</i>: ... elif <i>other</i>: ... else: ...</pre>	<pre>if (<i>condition</i>) { ... } else if (<i>other</i>) { ... } else { ... }</pre>
例	<pre>if x > 5: y = x ** 2 else: y = x * 2</pre>	<pre>if (x > 5) { y <- x ^ 2 } else { y <- x * 2 }</pre>

条件分岐（3項演算子）

項目	Python	R
構文	<code>x if condition else y</code>	<code>ifelse(condition, x, y)</code>
意味	条件が真なら x、偽なら y (式として値を返す)	ベクトル向けの条件付き選択 (長さに注意)
例	<code>"ok" if score >= 60 else "ng"</code>	<code>ifelse(score >= 60, "ok", "ng")</code>

* R: ifelse()。スカラーの if とは別物

関数定義 (def / function)

項目	Python	R
構文	<pre>def <i>name</i>(<i>args</i>): ... return ...</pre>	<pre><i>name</i> <- function(<i>args</i>) { ... return(...) }</pre>
例	<pre>def mean(xs): return sum(xs) / len(xs)</pre>	<pre>mean <- function(xs) { return(sum(xs) / length(xs)) }</pre>

パッケージ（モジュール）

import

- 使うときに読み込む必要がある
- コードの最初に1回だけ書くのが基本
- 例: `import pandas as pd`
- 例: `from pathlib import Path`
- 別名 `as` はコミュニティの慣例に従う (`pd, np, plt`)

名前空間・衝突

- どのパッケージの関数か分かりにくいときがある
- 同名関数があると後から `import` したものが有効になる
- パッケージ名を明示:
`pd.read_csv() / Path(...)`
- `from x import *` は避ける（名前空間が汚れる）
- R: `library(tidyverse) / readr::read_csv()`

パスとファイルの読み書き（標準モジュール）

項目	例	説明
pathlib.Path	<pre>from pathlib import Path p = Path("output") / "sample.txt"</pre>	パスをオブジェクトとして扱い「/」で部品を結合。
ディレクトリ作成	<pre>p.parent.mkdir(parents=True, exist_ok=True)</pre>	親ごと作成。既にあってもエラーにしない
書き込み（with句）	<pre>with p.open("w", encoding="utf-8") as f: f.write("hello\n")</pre>	ブロックを抜けると自動でclose
読み込み（with句）	<pre>with p.open("r", encoding="utf-8") as f: text = f.read() # 行ごと: for line in f:</pre>	全体なら read()、行ごとなら for
モード	<pre>"r" / "w" / "a"</pre>	読み込み / 書き込み（上書き） / 追記
文字コード	<pre>encoding="utf-8"</pre>	明示的に指定

演習問題 (Must)

- 問題 : [exercises/02_syntax_basics/python/](#)
 - 対話モードで以下の問題に解答していく。
 - 自分のコードを残したい人はファイルに転記しておく。
 - 01_objects.py
 - 02_control_flow.py
 - 03_collections.py
- 正解 : [exercises/02_syntax_basics/solutions/](#)

演習問題 (Stretch)

- 04_io.py: 問題1-3。out/ を作成し、with + pathlib で note.txt と mini.csv を書いて読み戻し表示する。

Exercise 3

文法（応用）

学習目標 (Must)

- 関数に処理を切り出して再利用できる
- クラスの最小形を読める (対訳の S3 も眺める)
- ログで実行状況を残せる
- Stretch: pytestで小さなテストを動かす

クラス

- オブジェクトの設計図（雛形）を定義したもの → オブジェクト指向
 - インスタンス: クラスを呼び出してオブジェクト化したもの（実物）
- クラスの構成要素
 - 属性 attribute: 値を割り当てたもの（括弧なし）
 - メソッド: 関数のようなもの（括弧あり）
 - 引数 self: メソッドの第1引数としてインスタンス自体が割り当てられる
- コンストラクター `__init__()`: インスタンスの初期設定（主に属性）をするメソッド

クラスを使う意義

- 関連するデータ（属性）と操作（メソッド）をひとまとめにできる
- 同じ設計図から複数のインスタンスを作れる（再利用）
- 継承で共通部分を親にまとめ、差分だけ子に書ける
- 大きなプログラムを部品（オブジェクト）に分けて考えやすい

メソッドと関数の違い

- 関数: 単独で呼べる処理。引数を受け取り、戻り値を返す
- メソッド: クラスに属し、インスタンスに紐づいて呼ばれる
 - 第1引数 `self` にそのインスタンス自身が渡される
 - 呼び出し: `obj.method(...)` $\leftrightarrow$ 関数は `func(...)`
- 属性は括弧なし (`obj.name`)、メソッドは括弧あり (`obj.method()`)

クラスの定義

- classキーワードで定義。（注：「class」は予約語！）

- 例：

```
class dummy_base:
    def __init__(self, attr1): # コンストラクター
        self.attr1 = attr1    # 属性（違う名前も可）

    def method1(self, var1):   # メソッド（関数と同様）
        return var1
```

- インスタンス化

- dummy = dummy_base(attr1=...)

演習イメージ: クラス（定義・属性・メソッド）

- クラスの基礎（定義、属性、メソッド）を理解する
- Must: 02_classes.py で Penguin / BandedPenguin を実装
- 対訳:
exercises/03_syntax_advanced/r/02_s3_class.R

クラスの継承

- 継承: 基となるクラス (基底クラス) の機能を変更・追加して新しいクラスを作ること
 - 親クラス (基底クラス) → 子クラス
 - 子クラスに共通の機能を親クラスに集約し、子は用途ごとの機能のみを持つ
- 定義: `class` キーワードの子クラス名の後ろに括弧で親クラス名を渡す

クラスの継承

- 例

```
class dummy_child(dummy_base):  
    def __init__(self, attr2):  
        self.attr2 = attr2  
    def method2(self, var2):  
        return var2 - self.attr2
```

- インスタンス化

```
dummy2 = dummy_child(attr2=1)
```

なぜログが必要か

- プログラムは途中で止まったり、想定外の入力で挙動が変わったりする。
- 画面に一瞬出たメッセージだけでは、あとから原因を追いきにくい。
- 簡易的には `print()` で確認してもよいが、本質的には `print()` だけでは不十分。
 - 重要度（情報・警告・エラー）を分けられない。
 - 出し分けや一括のオン／オフがしにくい。
 - 時刻付きでファイルに残しにくい（結果とデバッグが混ざる）。
- ログに残すと、いつ・何が起きたかを後から確認・共有できる。
 - 実行状況を残す（開始・警告・終了など）。

ログ (logging)

- 標準モジュール logging
 - loggerインスタンスを作成して使うのがおすすめ。
 - 例：

```
from logging import getLogger
logger = getLogger(__name__)
```
 - `logger.info(...)`などでログを出力する。
- レベル: `DEBUG < INFO < WARNING < ERROR`
 - 基準レベルがINFOのときはDEBUGレベルのログは出力されない。

* R: logger など

なぜテストが必要か

- 関数やクラスは、直したつもりでも別の箇所を壊すことがある。
- 手で何度も試すのは手間で、見落としも出やすい。
- テストを書いておくと、期待どおりかを自動で繰り返し確かめられる。
 - 合格・不合格・範囲外など、境界の振る舞いも固定できる。
- 変更が強くなり、安心してリファクタリングできる。

テスト (pytest)

- 小さな関数の振る舞いを自動で確かめる。
- 標準モジュールのunittestでもよいが、pytestがデファクトスタンダード
- pytest: テスト関数を収集して実行。
 - assert文で期待値と結果を比較。
- mock: 依存を差し替えて、その関数だけを単体で試せる。
 - 例: unittest.mock / pytest の monkeypatch

* R: testthat など

pytest の例

- テストする対象のスクリプトを読み込んで実行
 - 例 : `check_score.py`
- テストの実行
 - `uv run pytest` テストスクリプトへのパス

```
# test_pass_fail.py
from check_score import pass_fail

def test_pass():
    assert pass_fail(60) == "合格"

def test_invalid():
    with pytest.raises(ValueError):
        pass_fail(-1)
```

演習問題 (Must)

- `exercises/03_syntax_advanced/python/`
 - VS Codeで以下の問題ファイルに解答していく。
 - 実行は上記フォルダーに移動してから `uv run python 01_classes.py`
 - `01_classes.py`
 - `02_logging.py`
 - これは対話モードでも可。
 - `03_test_check_score.py`

Exercise 4

データ加工（基礎）

学習目標 (Must)

- 基本統計量と欠損を把握できる
- groupby で種・島ごとに集約できる
- Stretch: NumPy

メソッド連鎖

Method chaining (パイプに相当)

メソッド連鎖（Rのパイプ演算子相当）

メリット

- 値を左から右へ渡すイメージ。返り値に次のメソッドを続ける
- 入れ子 $f(g(h(x)))$ より、処理の流れが上から下へ読める
- カスタム処理は `.pipe(func)` で差し込める

デメリット

- 途中状態を見にくい（デバッグしづらいことがある）
- 長い連鎖は行が伸び、読む負担になる
- 副作用のある処理を挟むと意図が分かりにくい
- 必要なら中間変数に一度入れる

* https://pandas.pydata.org/docs/user_guide/basics.html#pipeline-using-pipe

メソッド連鎖の例

```
df.loc[
    df.loc[:, "Species_short"] == "Adelie",
    :
].assign(
    ratio=lambda d: d.loc[:, "Flipper_Length"] / d.loc[:, "Culmen_Length"]
).groupby(["Island"], observed=True).agg(
    mean_mass=("Body_Mass", "mean")
)
```

* 括弧で囲むと途中改行しやすい。R: %>% / |> (Ctrl/Cmd+Shift+M)

データ読み書き

Loading & saving data

with によるファイル読み書き

- with は文脈管理（context manager）の構文
- ブロックに入るときリソースを確保し、抜けるとき自動で解放する。
- 例外が起きても close される → 手動 f.close() 忘れを防ぐ。
- 形:
with 式 as 変数:
 処理
- ファイル読み書き以外にも使える（DB接続など）

* R: with() / on.exit()。表データは pandas 側が多い

データ読み書き

Python

- 表データの読込・保存は pandas + pyarrow が中心
- パスは pathlib.Path で扱うと安全
- 保存形式は parquet がおすすめ

R (紹介のみ)

- tidyverse: readr / readxl / arrow など
- library(tidyverse) でまとめて使えることが多い

データ読み込み

形式	Python	備考
CSV	<code>pd.read_csv()</code>	大規模データは他エンジン（例：pyarrow）も検討
Excel	<code>pd.read_excel()</code>	要 openpyxl。xlsx / xls 両対応（xls は非推奨）
parquet	<code>pd.read_parquet()</code>	
JSON	<code>pd.read_json()</code> / json モジュール	
SAS / SPSS / Stata	pyreadstat など	
Web	requests + pandas	Stretchの 05_io_db_web

* R: readr / readxl / arrow など

データ出力

- テーブルデータ（表形式）は parquet 形式がおすすめ！
- データ型が保存される / Python以外でも使えることが多い
- 圧縮率が高く、大規模データ向き
- 短所：言語独自の型は共通形式に落ちる場合がある
 - 例: `df.to_parquet("data/penguins.parquet")`
- CSV: `df.to_csv(...)` / Excel: `df.to_excel(...)`

* R: `arrow` / `write.csv` / `writexl` など

データ加工の基礎

Basic data wrangling

Pandas

- Pythonにおけるデータ加工のデファクトスタンダード
 - DataFrameを基本とした表形式データの処理
 - インデックスはRのデータフレームと大きく異なり最初は分かりにくい。
 - 最近Polarsのような競合も。
 - チートシート：
https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

		a	b	c
N	v			
D	1	4	7	10
	2	5	8	11
e	2	6	9	12

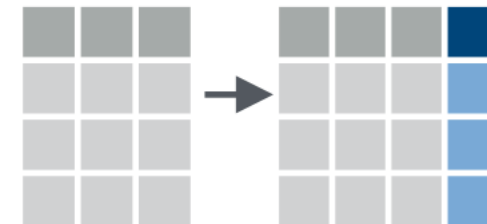
列の抽出



項目	Python	R
役割	列を選んで DataFrame / Series にする	dplyr::select / pull
1列	<code>df.loc[:, "col1"]</code>	<code>df > pull(col1)</code>
複数列	<code>df.loc[:, [, ["col1", "col2"]]</code>	<code>df > select(col1, col2)</code>

* 列の参照は `.loc[:, ...]` を使う

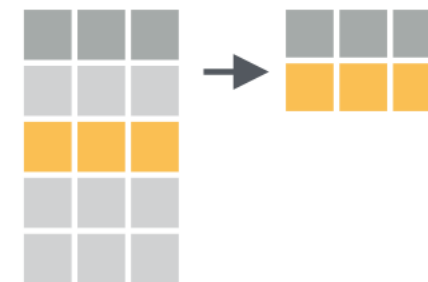
列の追加



項目	Python	R
役割	新しい列を追加した DataFrame を返す	dplyr::mutate
例	<pre>df.assign(col_ratio=lambda d: d.loc[:, "col1"] / d.loc[:, "col2"])</pre>	<pre>df > mutate(col_ratio = col1 / col2)</pre>

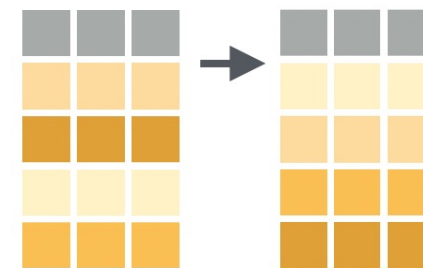
* assign は新しい DataFrame を返す

行の抽出



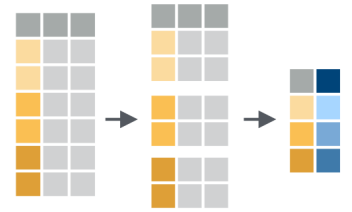
項目	Python	R
役割	条件に合う行だけを取り出す	dplyr::filter
例	<pre>df.loc[(df.loc[:, "col1"] == 1) & (df.loc[:, "col3"] == "A"), :]</pre>	<pre>df > filter(col1 == 1, col3 == "A")</pre>

行のソート



項目	Python	R
役割	指定列を基準に並べ替える（デフォルトは昇順）	dplyr::arrange
例	<pre>df.sort_values("col1") df.sort_values(["col1", "col2",], ascending=[True, False])</pre>	<pre>df > arrange(col1) penguins > arrange(col1, desc(col2))</pre>

グループごとの集約



項目	Python	R
役割	キーごとに件数・平均などを集約する	group_by + summarise
例	<pre>df.groupby(["col2", "col3"], as_index=False, observed=True,) .agg(n=("col1", "count"), avg=("col1", "mean"))</pre>	<pre>df > group_by(col2, col3) > summarise(n = sum(!is.na(col1)), avg = mean(col1, na.rm = TRUE))</pre>

* Excelのピボット ↔ groupby + agg

この他にも…

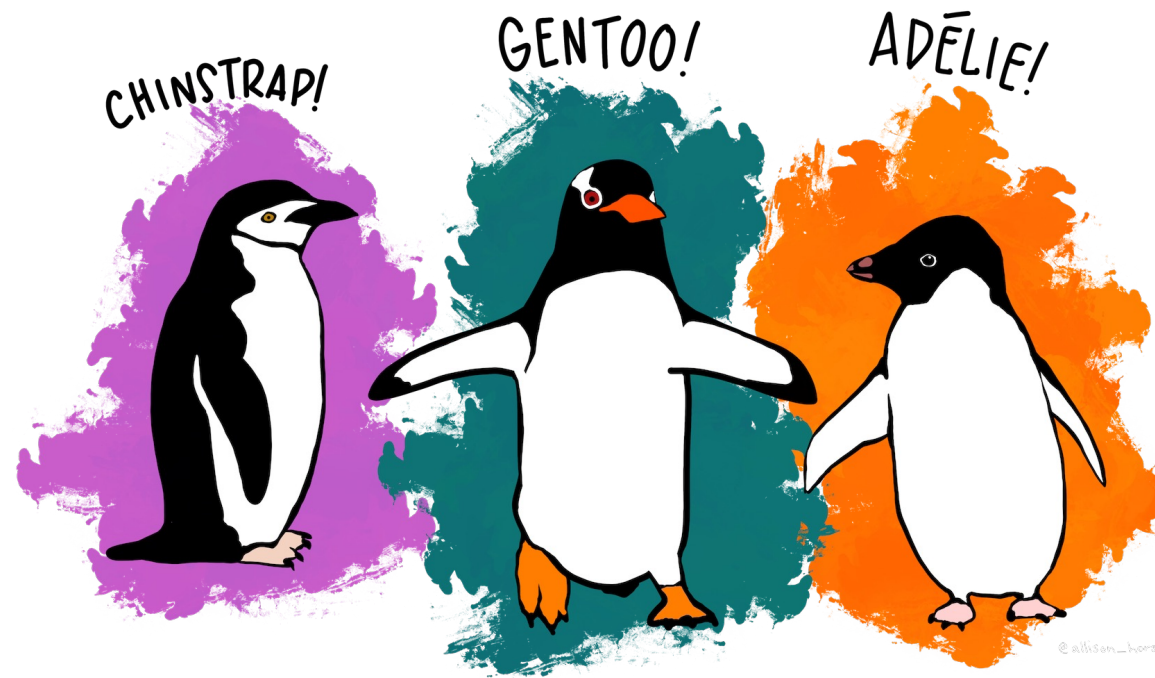
項目	Python	R
重複削除	<code>df.drop_duplicates()</code>	<code>dplyr::distinct</code>
縦結合	<code>pd.concat([df1, df2], axis=0)</code>	<code>bind_rows</code> / <code>UNION</code>
横結合	<code>pd.concat([df1, df2], axis=1)</code>	<code>bind_cols</code>
列の削除	<code>df.drop(columns=["col1"])</code>	<code>dplyr::select(-col1)</code>

大きな結合・集約は SQL も検討

- 結合（JOIN）やグループ集約は、データが大きいほどDB側で処理した方が速いことが多い。
- ライブラリー
 - Python: `pd.read_sql(query, conn)` / `df.to_sql(...)`
 - R: `DBI + dbGetQuery()` / `dbWriteTable()`

演習：ペンギンデータ

- 2007–9年、南極パーマー基地周辺の調査データ
- DOI: <https://doi.org/10.1371/journal.pone.0090081>



* <https://allisonhorst.github.io/palmerpenguins/articles/art.html>

ペンギンデータの主な列（変換後）

列名	意味
Species_short	ペンギンの種（短縮）
Island	データを収集した島の名前
Individual_ID	個体識別番号（異種間では重複あり）
Date_Egg	調査年月日（巣に卵が確認された日）
Culmen_Length / Depth	くちばしの長さ / 太さ（単位：mm）
Flipper_Length	ひれの長さ（単位：mm）
Body_Mass	体重（単位：g）
Sex	性別

データ取得手順 (01_fetch_penguins)

- 1. Rdatasets の Available datasets を開く
 - <https://vincentarelbundock.github.io/Rdatasets/articles/data.html>
- 2. Search で penguins を検索し、Package = datasets の行を選ぶ
- 3. Item = penguins_raw (penguins) の CSV を data/penguins_raw.csv に置く

手順1: Available datasets を開く

- Rdatasetsにアクセス。
 - <https://vincentarelbundock.github.io/Rdatasets/articles/data.html>

Rdatasets 1.0.0 Available datasets

Source: vignettes/data.Rmd

Search:

	Package	Item	Title	CSV	Doc	Rows	Cols	n_binary	n_character
1	admiral	admiral_adlb	Lab Analysis Dataset	CSV	Doc	330	115	13	79
2	admiral	admiral_adsl	Subject Level Analysis Dataset	CSV	Doc	306	54	5	39
3	admiral	atoxgr_criteria_ctcv4	Metadata Holding Grading Criteria for NCI-CTCAEv4 using SI unit where applicable	CSV	Doc	42	13	2	13
4	admiral	atoxgr_criteria_ctcv4_uscv	Metadata Holding Grading Criteria for NCI-CTCAEv4 using USCV unit where applicable	CSV	Doc	48	13	2	13
5	admiral	atoxgr_criteria_ctcv5	Metadata Holding Grading Criteria for NCI-CTCAEv5 using SI unit where applicable	CSV	Doc	40	13	2	13
6	admiral	atoxgr_criteria_ctcv5_uscv	Metadata Holding Grading Criteria for NCI-CTCAEv5 using USCV unit where applicable	CSV	Doc	46	13	2	13
7	admiral	atoxgr_criteria_ctcv6	Metadata Holding Grading Criteria for NCI-CTCAEv6 using SI unit where	CSV	Doc	43	13	2	13

手順2: penguins を検索する

- Search に penguins と入れる
- Package = datasets の行を選ぶ（他パッケージの同名もある）
- Item = penguins_raw (penguins) を使う（17列の raw）

Rdatasets 1.0.0 Available datasets

Source: vignettes/data.Rmd

Search:

	Package	Item	Title	CSV	Doc	Rows	Cols	n_binary	n_character	n_factor	n_logical	n_numeric
595	bayesrules	penguins_bayes	Penguins Data	CSV	Doc	344	9	2	0	4	0	5
1059	datasets	penguins	Measurements of Penguins near Palmer Station, Antarctica	CSV	Doc	344	8	1	0	3	0	5
1060	datasets	penguins_raw (penguins)	Measurements of Penguins near Palmer Station, Antarctica	CSV	Doc	344	17	2	9	0	0	7
1728	heplots	peng	Size measurements for penguins near Palmer Station, Antarctica	CSV	Doc	333	8	1	0	3	0	5
2189	modeldata	penguins	Palmer Station penguin data	CSV	Doc	344	7	1	0	3	0	4
			Size measurements for adult									

手順3: CSV を data/ に置く

- CSV リンクからダウンロードする
- 保存先:
data/penguins_raw.csv
- その後
01_fetch_penguins.py
で整形し
penguins.parquet へ

```
penguins_raw.csv → 保存先: data/penguins_raw.csv
CSV リンクからダウンロードし、リポジトリの data/ に置く

rownames,studyName,Sample Number,Species,Region,Island,Stage,Individual ID,Clutch Completion,Date Egg,Culmen Length (mm),Culmen Depth (mm),Flipper Len
1,PAL0708,1,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N1A1,Yes,2007-11-11,39.1,18.7,181,3750,MALE,,Not enough blood f
2,PAL0708,2,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N1A2,Yes,2007-11-11,39.5,17.4,186,3800,FEMALE,8.94956,-24.69454,
3,PAL0708,3,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N2A1,Yes,2007-11-16,40.3,18.195,3250,FEMALE,8.36821,-25.33302,
4,PAL0708,4,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N2A2,Yes,2007-11-16,,,,,,Adult not sampled.
5,PAL0708,5,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N3A1,Yes,2007-11-16,36.7,19.3,193,3450,FEMALE,8.76651,-25.32426,
6,PAL0708,6,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N3A2,Yes,2007-11-16,39.3,20.6,190,3650,MALE,8.66496,-25.29805,
7,PAL0708,7,Adelie Penguin (Pygoscelis adeliae),Anvers,Torgersen,"Adult, 1 Egg Stage",N4A1,No,2007-11-15,38.9,17.8,181,3625,FEMALE,9.18718,-25.21799,
...
```

* 直リンク: https://vincentarelbundock.github.io/Rdatasets/csv/datasets/penguins_raw.csv

演習問題 (Must)

- `exercises/04_wrangling_basics/python/`
 - Jupyter Notebook : R Markdownのようにコードと実行結果を保持。
 - VS Codeで以下の問題ファイルに解答していく。
 - `01_read_describe.jpynb`
 - `02_groupby.jpynb`
 - `03_duplicates.jpynb`

NumPyの基礎

トピック	主なAPI・要点
配列の作り方	zeros / ones / full / arange / linspace / eye · dtype·astype
参照・転置	A[i,j]、行 A[0]、列 A[:,1]、スライス A[:2,:2]、転置 A.T
reshape	reshape / ravel (行優先 C 順。order="F" で列優先)
対角 (diagonal)	np.diag(A) で取り出し、np.diag(v) で対角行列
演算と軸	要素ごとの四則、sum / mean / std / min·max (axis) · cumsum
ブロードキャスト	形の違う配列どうしの演算。足りない軸は1として揃えて計算
条件とマスク	比較演算 → ブール索引 (A[A>6] など) 、 np.where
行・列の追加 (stack)	np.hstack / np.vstack / concatenate (axis=0/1)
少し先の基本	@ (行列積) 、 argsort / ソート、 unique、 split

演習問題 (Stretch)

- `exercises/04_wrangling_basics/python/`
 - VS Codeで以下の問題ファイルに解答していく。
 - `04_numpy_basics.jpynb`

Exercise 5

データ加工（応用）

学習目標（Must）

- 複数のデータセットをキーで結合できる。
- データセットの縦持ち／横持ちを変換できる。

tidyなデータ

- 条件
 - 各**変数**が1つの列で構成されている
 - 各**観測**が1つの行で構成されている
 - 各**観測単位**が1つの表で構成されている（各**値**が1つのセルに入っている）
- 必ずしも縦長なデータとは限らない！
- 実務と折り合いをつける！
 - 結局のところ**使いやすさ**。

country	year	cases	population
Afghanistan	1999	181	19997071
Afghanistan	2000	1866	20595360
Brazil	1999	31737	172106362
Brazil	2000	81488	174504898
China	1999	211258	1272015272
China	2000	216766	1280125583

variables

country	year	cases	population
Afghanistan	1999	181	19997071
Afghanistan	2000	1866	20595360
Brazil	1999	31737	172106362
Brazil	2000	81488	174504898
China	1999	211258	1272015272
China	2000	216766	1280125583

observations

country	year	cases	population
Afghanistan	1999	181	19997071
Afghanistan	2000	1866	20595360
Brazil	1999	31737	172106362
Brazil	2000	81488	174504898
China	1999	211258	1272015272
China	2000	216766	1280125583

values

* <https://r4ds.had.co.nz/tidy-data.html#fig:tidy-structure>

縦持ち (melt)

table4a

country	1999	2000
A	0.7K	2K
B	37K	80K
C	212K	213K



country	year	cases
A	1999	0.7K
B	1999	37K
C	1999	212K
A	2000	2K
B	2000	80K
C	2000	213K

Python

```
table4a.melt(  
    id_vars="country",  
    var_name="year",  
    value_name="cases",  
)
```

R

```
table4a |>  
  pivot_longer(  
    cols = 2:3,  
    names_to = "year",  
    values_to = "cases"  
  )
```

横持ち (pivot_table)

table2

country	year	type	count
A	1999	cases	0.7K
A	1999	pop	19M
A	2000	cases	2K
A	2000	pop	20M
B	1999	cases	37K
B	1999	pop	172M
B	2000	cases	80K
B	2000	pop	174M
C	1999	cases	212K
C	1999	pop	1T
C	2000	cases	213K
C	2000	pop	1T

→

country	year	cases	pop
A	1999	0.7K	19M
A	2000	2K	20M
B	1999	37K	172M
B	2000	80K	174M
C	1999	212K	1T
C	2000	213K	1T

Python

```
table2.pivot_table(  
    index=["country", "year"],  
    columns="type",  
    values="count",  
)
```

※indexに指定したキーに**重複があると集約**されるので注意！

R

```
table2 |>  
  pivot_wider(  
    names_from = "type",  
    values_from = "count"  
  )
```

この他にも…

項目	Python	R
欠測値補完	<code>df.fillna()</code>	<code>tidyr::fill / replace_na</code>
欠測値のある行を除く	<code>df.dropna()</code>	<code>tidyr::drop_na</code>
列を分割	<code>df.loc[:, "col"].str.split()</code> <code>df.loc[:, "col"].str.extract()</code>	<code>tidyr::separate / separate_wider_delim / separate_longer_delim</code> など

キー結合 (merge)

項目	Python	R
役割	キー列で表を結合する (SQL JOIN)	dplyr::left_join など
例	<pre>penguins.merge(island_master, on="Island", how="left",)</pre>	<pre>penguins > left_join(island_master, by = "Island")</pre>
結合の種類	"left" / "inner" "right" / "outer"	left_join / inner_join right_join / full_join

* 結合後は行数が増減しうる → 前後の件数を確認。キーの型・表記ゆれに注意

カテゴリー値の扱い

項目	Python (Pandas)	R
意味	文字列に見えるが「取り得る水準」を持つ型（文字列とは別のデータ型）	factor（水準 levels を持つ）
作る・変換	<code>df.loc[:, "col"].astype("category")</code> <code>pd.Categorical(x, categories=[...])</code>	<code>factor(x) / as.factor(x)</code> <code>factor(x, levels = c(...))</code>
順序あり	<code>pd.Categorical(x, categories=[...], ordered=True)</code>	<code>factor(x, levels = c(...), ordered = TRUE)</code>
メリット	メモリ節約・水準の明示・棒グラフ等の並び	同様（levels で軸の順を制御）

カテゴリー値とgroupby

項目	Python	R
落とし穴	未出現の水準も行に出る（件数0など）	levels に残ると同様に空グループが出うる
対処	<code>.groupby("Island", observed=True)</code> # 出現した水準だけ集約	<code>droplevels(x)</code> # 名前のみ: <code>forcats::fct_drop</code>
注意点	データの結合や集約時	結合前に不要な levels を落とす

* Ex4 の groupby 例でも observed=True。join 前の集約でも同じ注意

画像データ (Pillow + NumPy)

```
from PIL import Image
import numpy as np
```

```
im = Image.open("data/4.2.07.tiff")
print(im.format, im.size, im.mode)  # TIFF (512, 512) RGB
im.show()                          # 表示
```

```
arr = np.array(im)                  # shape (H, w, C)
crop = arr[0:330, 0:280, :]         # 左上を切り取り
b = arr[:, :, 2]                   # Bチャンネル
print(b.shape)                      # サイズ確認
```



* <https://sipi.usc.edu/database/database.php>

演習問題 (Must)

- `exercises/05_wrangling_advanced/python/`
 - VS Codeで以下の問題ファイルに解答していく。
 - `01_join.jpynb`
 - `02_pivot.jpynb`

演習問題 (Stretch)

- `exercises/05_wrangling_advanced/python/`
 - VS Codeで以下の問題ファイルに解答していく。
 - `03_image_array.jpynb`

Exercise 6

データ可視化

学習目標 (Must)

- 5種類（散布・折れ線・棒・ヒストグラム・箱ひげ）とファセットをPlotlyで扱える。
- CUDに配慮したグラフを描ける。

グラフの種類

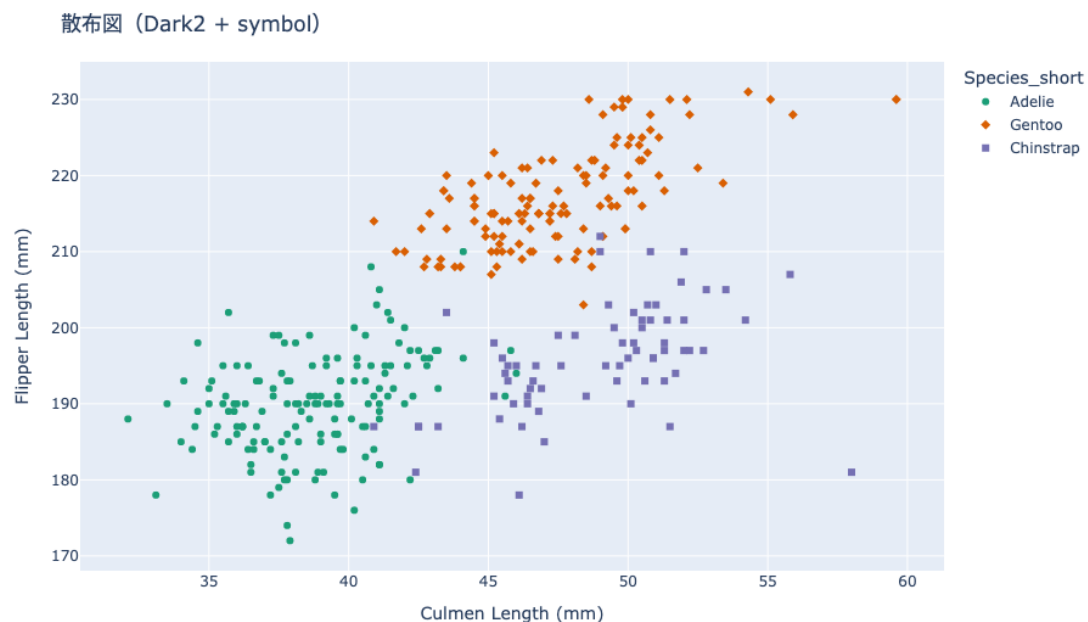
種類	用途	Plotly	ggplot2
散布図	2量的変数の関係	<code>px.scatter()</code>	<code>geom_point()</code>
折れ線	時系列の推移	<code>px.line()</code>	<code>geom_line()</code>
棒グラフ	カテゴリの比較	<code>px.bar()</code>	<code>geom_col()</code>
ヒストグラム	1変数の分布	<code>px.histogram()</code>	<code>geom_histogram()</code>
箱ひげ図	ばらつき・四分位	<code>px.box()</code>	<code>geom_boxplot()</code>

- 考慮すべきこと：CUD（カラーユニバーサルデザイン）
 - 人によって色の見え方が違う
 - 色だけでなく形・線種・模様も変える（Dark2 + symbol / line_dash / pattern_shape）

* R（ggplot2）のCUD: colour/fill に加え shape・linetype、必要なら模様（ggpattern）。

散布図

- Plotly: `px.scatter()` + `color` + `symbol`
- ggplot: `geom_point()` + `scale_color_brewer()`
- CUD: 色とマーカー形を併用



散布図（例：ひれの長さと体重）

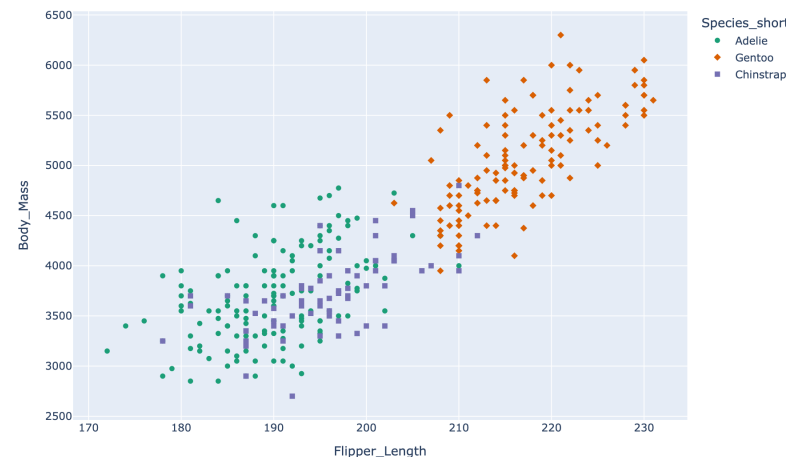
Plotly

```
px.scatter(  
    df,  
    x="Flipper_Length",  
    y="Body_Mass",  
    color="Species_short",  
    color_discrete_sequence=  
        px.colors.qualitative.Dark2,  
    symbol="Species_short",  
)
```

※この図を表示するには次のようにする：
fig = px.scatter(...)
fig.show()

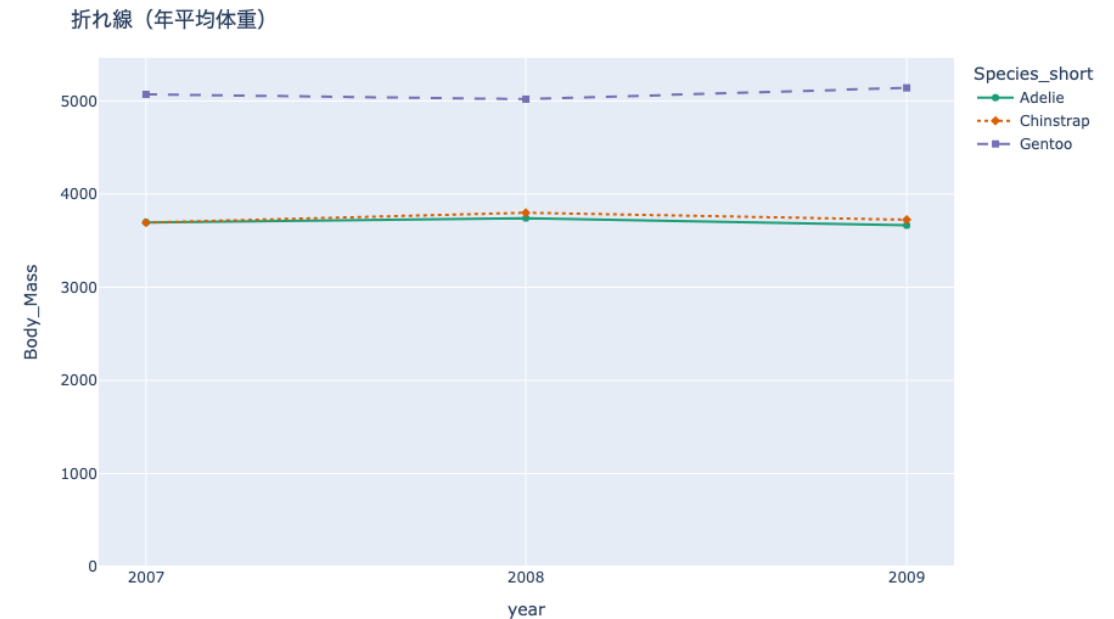
ggplot2

```
ggplot(  
    df,  
    aes(  
        x = flipper_len,  
        y = body_mass,  
        colour = species,  
        shape = species)  
    ) +  
    geom_point() +  
    scale_color_brewer(palette = "Dark2")
```



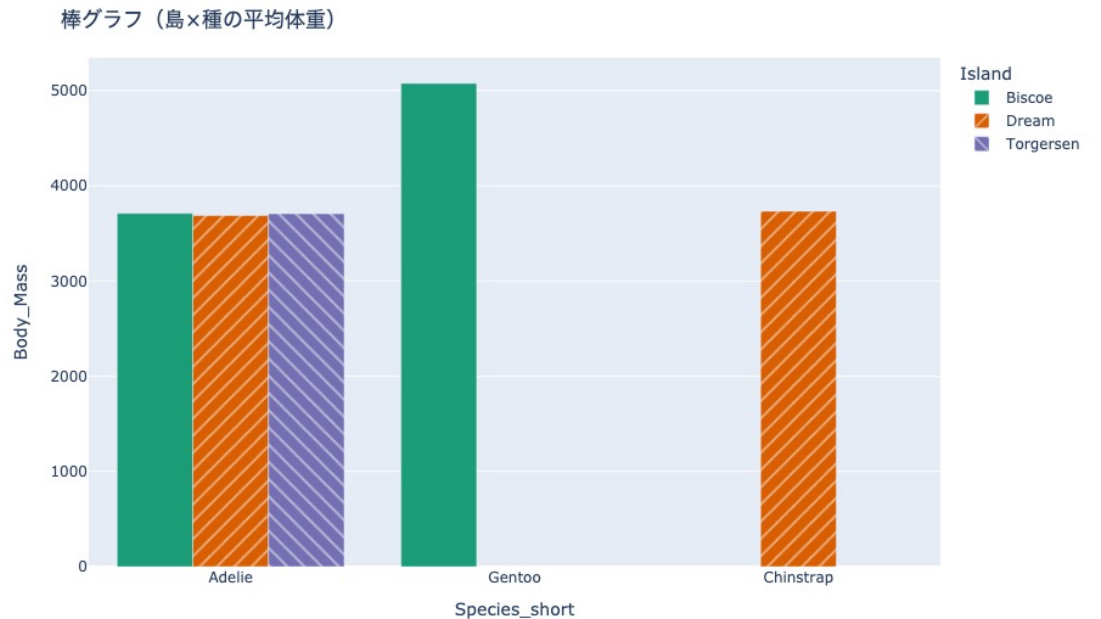
折れ線グラフ

- Plotly: `px.line()` + `line_dash` + `symbol`
- ggplot: `geom_line()` + `geom_point()` + `linetype`
- 縦軸は0始まりにする
 - `fig.update_yaxes(rangemode="tozero")`を追加



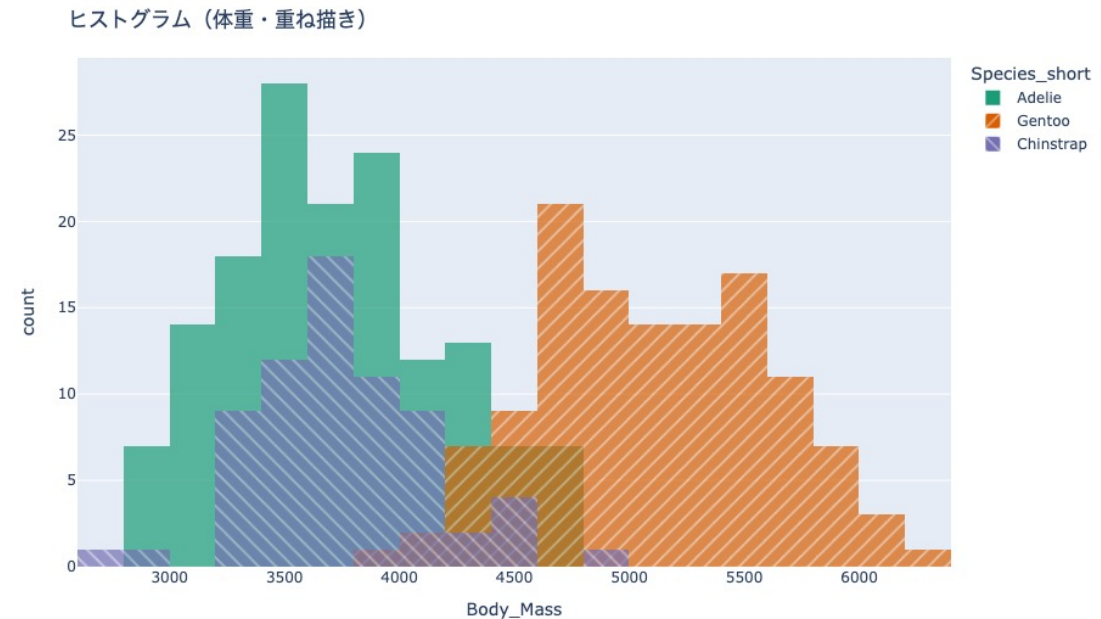
棒グラフ

- Plotly: `px.bar()` + `pattern_shape`
- ggplot: `geom_col()` + 模様/facet
- barmode
 - "group" : 並べる
 - "stack" : 積み上げ



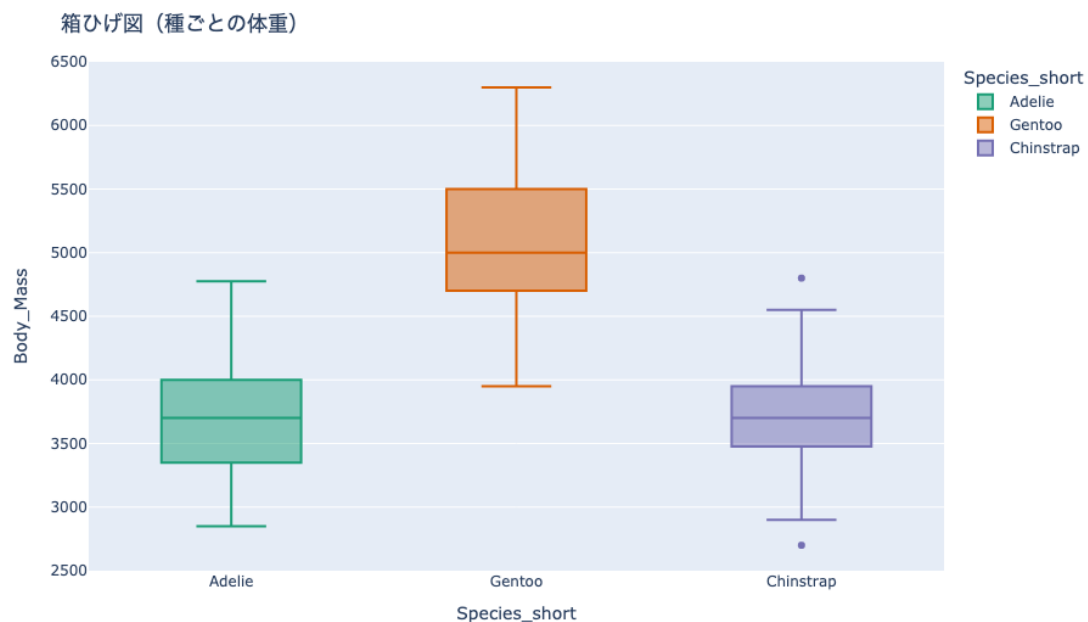
ヒストグラム

- Plotly: `px.histogram()` + `pattern_shape`
- ggplot:
`geom_histogram(alpha=...)`
- 不透明度 (ggplot2の`alpha`)
 - 完全に不透明 → `opacity=1`
 - 完全に透明 → `opacity=0`
- 重ね書き
 - `barmode="overlay"`



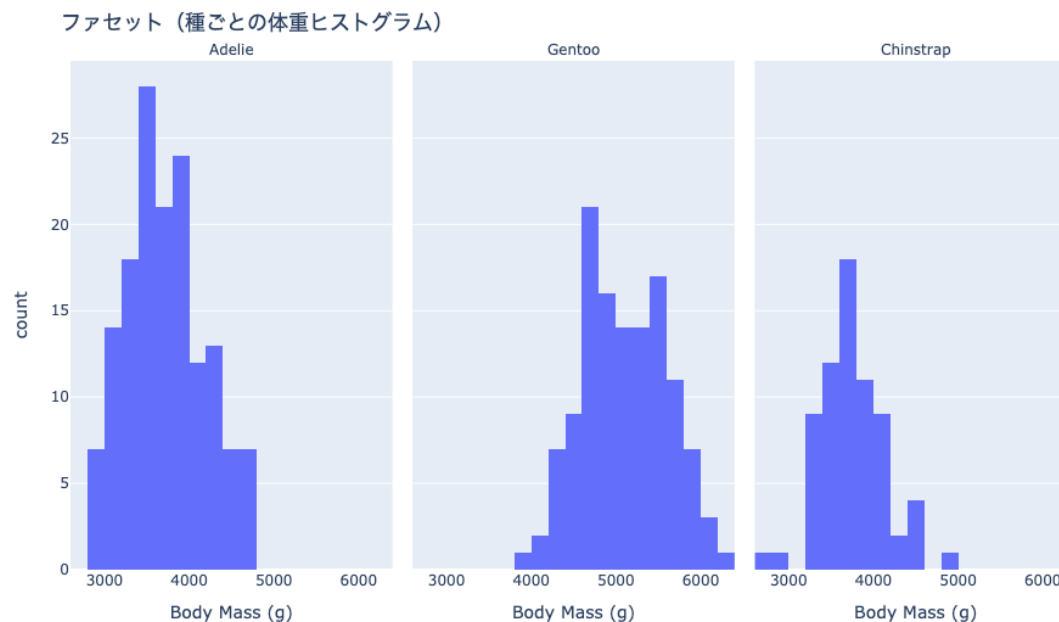
箱ひげ図

- Plotly: `px.box()`
- ggplot: `geom_boxplot()`
- 四分位・ひげ・外れ値の目安
- 群ごとのばらつき比較に便利



ファセット

- カテゴリごとに同じ図を並べる
(小画面分割)
- Plotly: `facet_col` / `facet_row`
- ggplot: `facet_wrap()` / `facet_grid()`



演習問題 (Must)

- `exercises/06_visualisation/python/`
 - VS Codeで以下の問題ファイルに解答していく。
 - `01_scatter.ipynb`
 - `02_line.ipynb`
 - `03_bar.ipynb`
 - `04_histogram.ipynb`
 - `05_box.ipynb`

学習目標 (Must)

- アンスコムをワイド→縦持ちに整形し、Plotly で描き、要約統計を計算できる
- Welchの t 検定を実行し、群平均とあわせて解釈できる
- 線形回帰 (statsmodels OLS) を動かせる
- Stretch: 分布・外れ値の要約 (03_distributions)